Data Structures and Algorithms
(CS221)

# Linked List

# Linked List

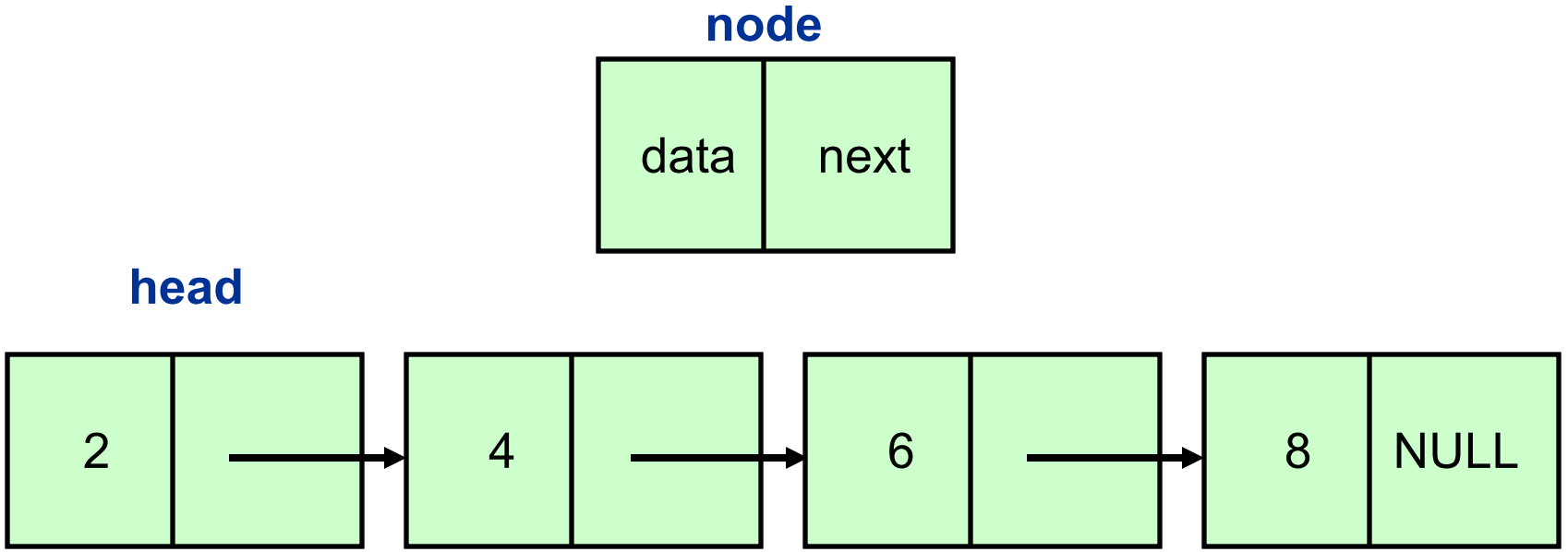
A linear data structure in which elements (nodes) are stored in memory **non-contiguously**

Each node consists of at least two types of elements

- An element to store some data/information
- A pointer to the next node in the list

- Usually the first node is known as the **header node**
- The pointer for the last node is usually a null pointer

# Linked List



# Linked List

```
struct node
      {
      int data;
      node *next;
      };
node *head;
```

**int data;**

- This is an integer variable that stores the actual value of the node.

**node *next**

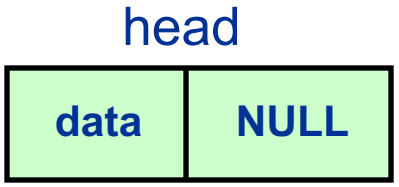
- This is a pointer to another node. It is used to link the current node to the next node in the list.
- If a node is the last node in the list, next will be NULL

# Linked List: Add an element at the head node

## Approach 1:

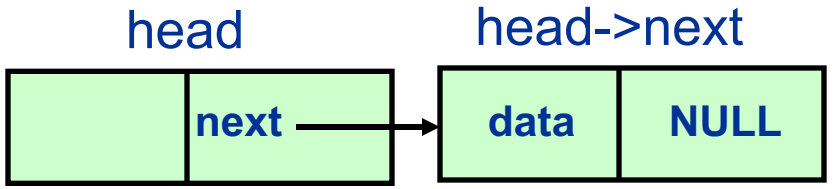
**Complexity : O(*1*)**

```
head = new node;
cin>>head->data;
head->next=NULL;
```



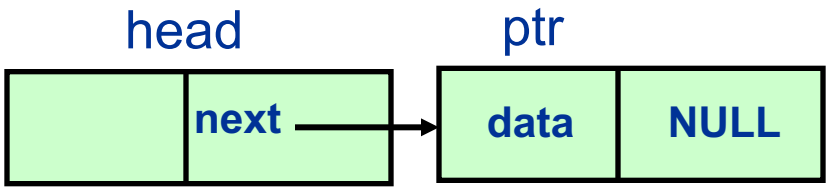

## Approach 2:

```
head->next = new node;
cin>> head->next ->data;
head->next->next=NULL;
```




OR

```
node *ptr =new node;
cin>> ptr ->data;
head->next=ptr;
ptr->next=NULL;
```



# Linked List: Add an element at the head node

**Approach 2:**
head node only serves as a global pointer to the start of the linked list
head node does not contain any data

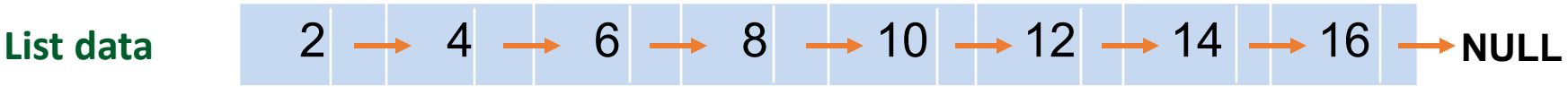
```
head->next = new node;
cin>> head->next ->data;
head->next->next=NULL;
```

OR

```
node *ptr =new node;
cin>> ptr ->data;
head->next=ptr;
ptr->next=NULL;
```

# Linked List: Traverse through the linked list

- Visit each element of the list
  - e.g., print each element of the list on the screen

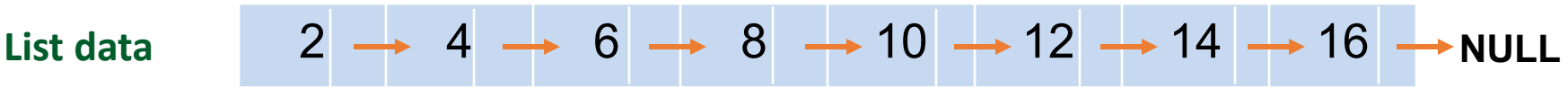



```
for (node *ptr = head; ptr != NULL; ptr = ptr->next)
    cout<< ptr->data;
```

Complexity : O(*N*)

# Linked List: Search for an element in the list

- Search for a specific element in the list
  - e.g., search and return the pointer to the element containing data=10, if found in the list




```
SearchElement = 10;
for (node *ptr = head; ptr != NULL; ptr = ptr->next)
        {
        if (ptr->data == SearchElement){
            indexPtr = ptr;
             return indexPtr;
         }
}
  indexPtr = NULL;
  cout<<   "Element not found"  ;
  return indexPtr;
```

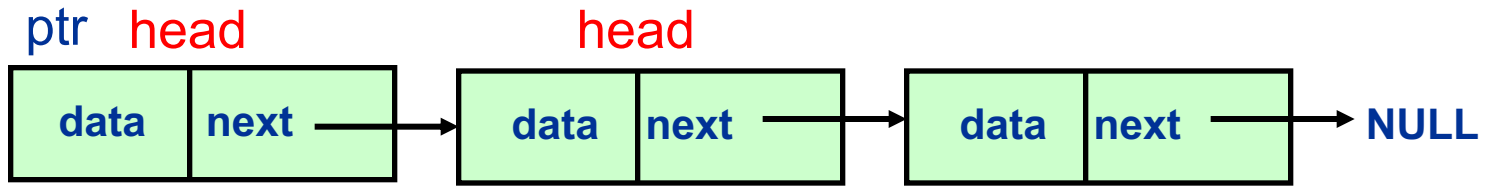
Complexity : O(*N*)

# Linked List: Append a new element before/at the head node

- head node contains data as well as a global start pointer

```
node *ptr = new node;
cin>> ptr ->data;
ptr->next = head;
head = ptr;
```

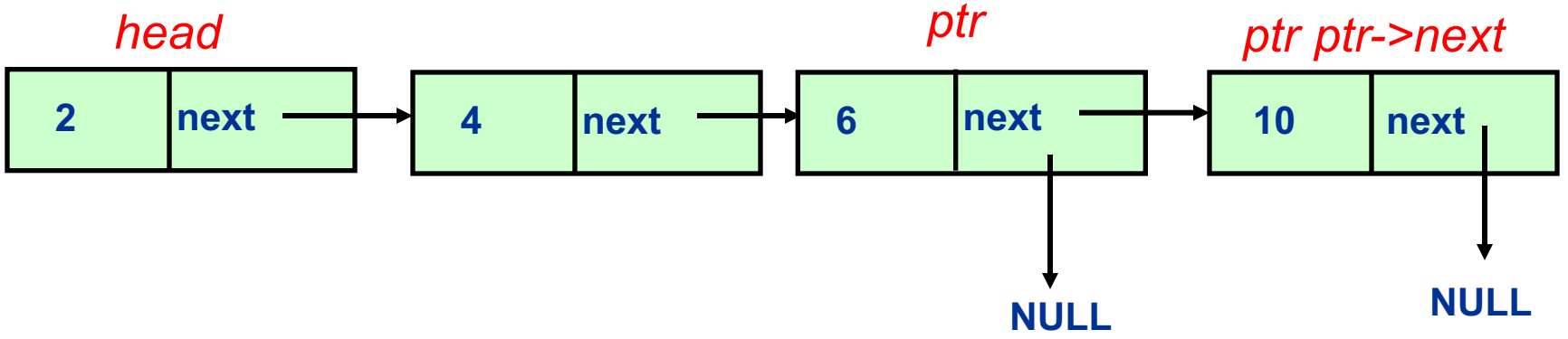
**Complexity : O(*1*)**



# Linked List: Append a new element after the tail node

- head node contains data as well as a global start pointer

```
node *ptr;
for (ptr = head; ptr->next != NULL; ptr = ptr->next){}
    ptr->next = new node;
      ptr = ptr->next;
      ptr->data = NewData; // e.g NewData =10;
      ptr->next = NULL;
```



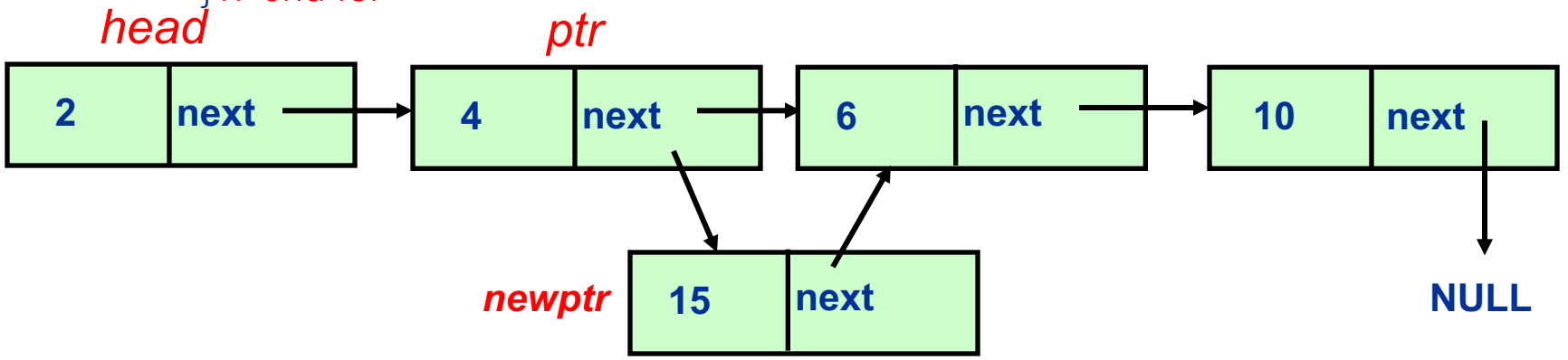

**Complexity : O(*N*)**

# Linked List: Insert a new element after a specific node

- Suppose you want to insert after the node with data = '*afterValue*'

```
node* newptr;
for(node *ptr=head; ptr!=NULL;ptr=ptr->next){
  if(ptr->data == afterValue)    //   e.g afterValue =4;
    {
    newptr=new node;
    newptr->data=NewData;  //   e.g NewData =15;
    newptr->next=ptr->next;
    ptr->next=newptr;
    } // end if
  } // end for
```

**Complexity : O(*N*)**

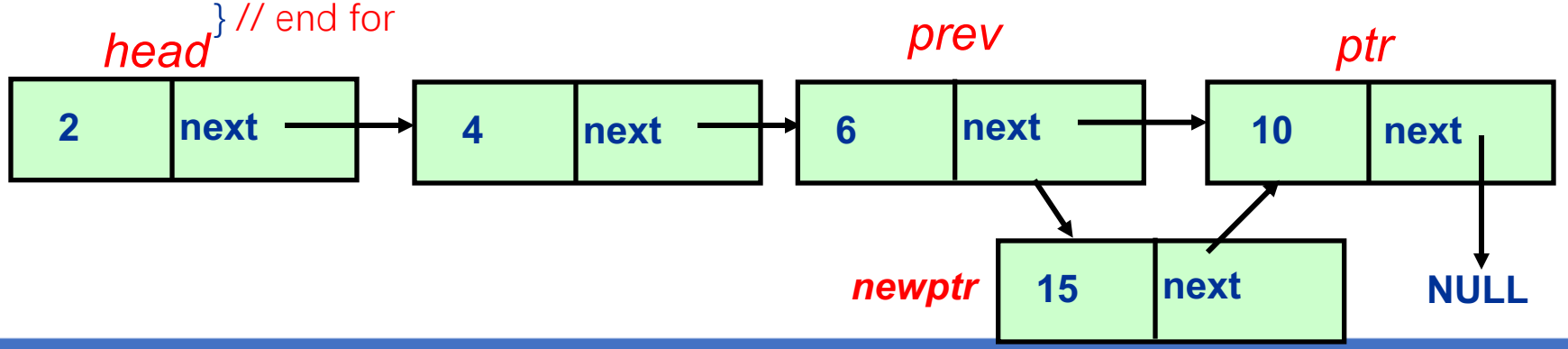


# Linked List: Insert a new element before a specific node

- Suppose you want to insert before the node with data = '*beforeValue*'

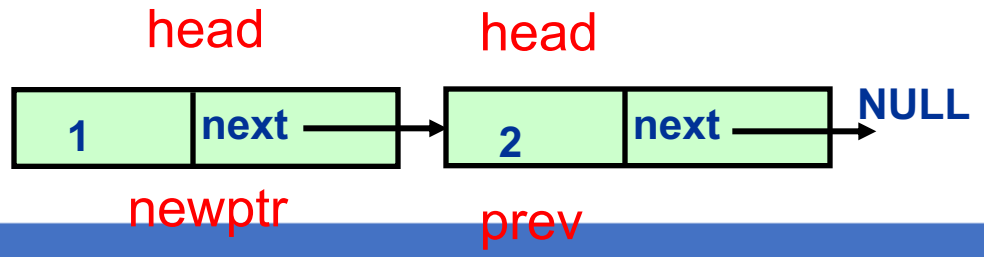
```
node* newptr;
  node *prev=head;
  for(node *ptr=head->next; ptr!=NULL; ptr=ptr->next){
    if(ptr->data == beforeValue)    //  e.g beforeValue =10;
      {
      newptr=new node;
      newptr->data=NewData;     //  e.g NewData =16;
      prev->next=newptr;
      newptr->next=ptr;
      } // end if
    prev=prev->next;
    } // end for
```

**Complexity : O(*N*)**



# Linked List: Insert a new element before a specific node (Only one node is currently there)

```
node* newptr;
 node *prev=head;
if(head->data == beforeValue) {    //  e.g beforeValue =2;
      newptr=new node;
      head=newptr;
      head->next=prev;
      newptr->data=NewData;    //   e.g NewData =1;
      return;
      }
```

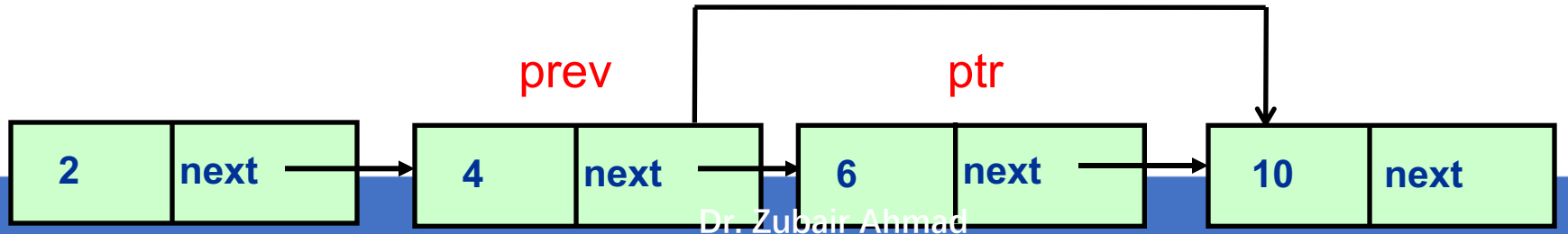


# Linked List: Delete an element from the list

- Suppose you want to delete the node with data = '*ValueToDelete*'

```
node *prev=head;
for(node *ptr=prev->next; ptr!=NULL; ptr=ptr->next){
if(ptr->data == ValueToDelete) // e.g ValueToDelete = 6;
  {
  prev->next=ptr->next;
  delete(ptr);
  return;
  }
  prev=prev->next;
}
```

**Complexity : O(*N*)**

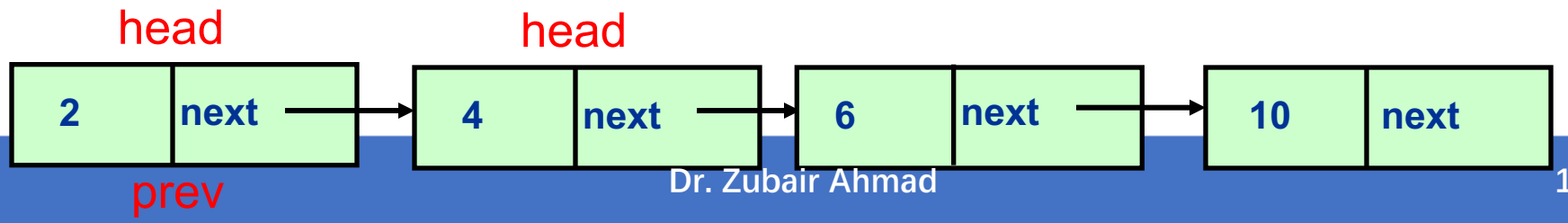


# Linked List: Delete an element from the list

- **Delete the head node from the list**

```
node *prev=head;
 if(head->data == ValueToDelete)
  {
  head=head->next;
  delete(prev);
    return;
  }
```

**Complexity : O(1)**




# Questions?

zahmaad.github.io